

Federated Graphql Migration- Product Packet

Problem Statement

What getTechPackResourceCountMap does

The **Tech Pack panel** is a summary view showing a business partner everything visible to them on a product. The query getProductTechPackCountV1 calls getTechPackResourceCountMap which returns IDs of resources grouped by type:

contract	
Input:	productId, partnerId, workspaceContext, parentProductId
Output:	{ productAttachments: [IDs] critical product packet files discussionAttachments: [IDs] attachments on critical discussions discussions: [IDs] critical discussions sample: [IDs] approved samples measurementSets: [IDs] claims: [IDs] productBoms: [IDs] packagingBoms: [IDs] constructions: [IDs] watchlists: [IDs] }

The UI uses these ID lists to render badge counts and navigation links in the Tech Pack panel.

Why this is hard to migrate

Problem	Why it matters
Full graph traversal	Calls getFilteredResourceAndLevelWiseChildrenBasedOnPartnerACLPermission with maxDepth=null — loads the entire product subtree from the Relationship Service into memory
Serial ACL checks	Every node in the tree gets an ACL permission check — N nodes = N calls. Latency scales with product complexity
Double traversal	When parentProductId is set, the entire process repeats for the parent product — doubling cost
8 domain types in one function	Attachments, discussions, samples, BOMs, claims, measurements, constructions, watchlists — all computed in a single 200-line function
Mixed data sources	Some data from the Relationship Service graph, some from Elastic Search, some from ACL service — inconsistent and fragile
Tight coupling	Any change to this function risks breaking multiple domain features. No domain team can own or deploy independently

The goal

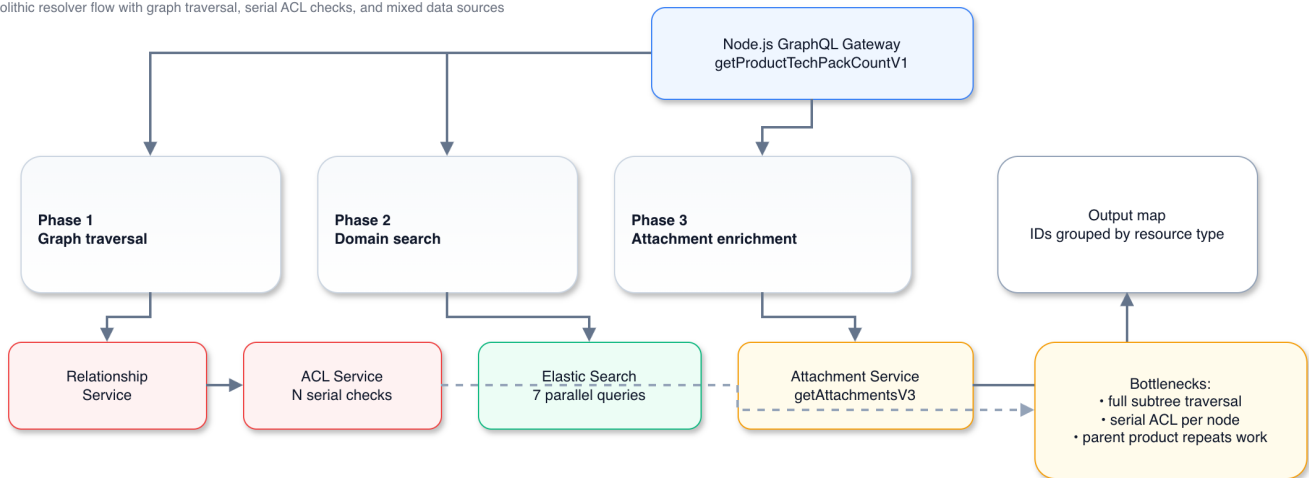
Replace this monolithic function with a federated GraphQL architecture where: - Each domain team owns its own data and resolver - No Relationship Service dependency - Partner visibility is a simple WHERE partner_id = ? clause - Each domain query runs in parallel

Current Architecture

How it works today (monolith)

Current Architecture

Monolithic resolver flow with graph traversal, serial ACL checks, and mixed data sources



Key data flow (step by step)

1. Call Relationship Service get all child nodes of product (sample, discussions, discussionThreads, bill_of_materials, packaging_bom, claim, measurement_set, construction_set, product_watchlist)
2. For each node ACL check: "Can partner X see this node?"
Remove nodes the partner cannot see.
3. If parentProductId exists repeat steps 1-2 for parent product.
4. Collect attachment IDs from the graph (depth levels 0-3).
Fetch full attachment objects via getAttachmentsV3.
Filter for product_packet_props.critical=true AND partner_id match.
5. Run 7 parallel Elastic Search queries for domain-specific counts: samples, criticalDiscussions, measurementSets, claims, boms, constructions, watchlists.
6. Process critical discussions extract criticalDiscussionIds + parentDiscussionIds (from threads) fetch their attachments.
7. Build outputMap with all ID lists return.

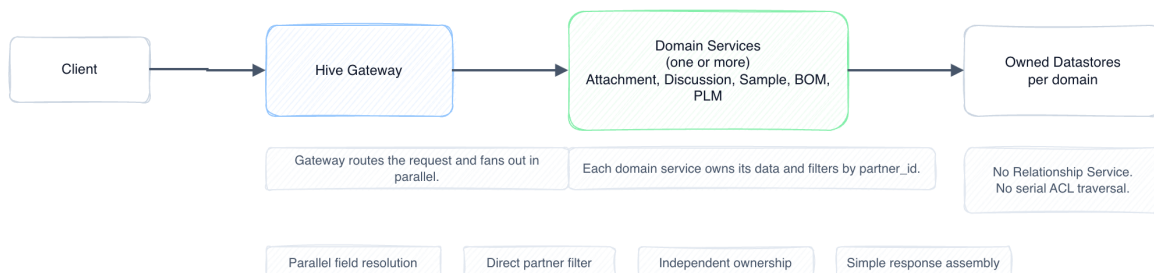
Target Architecture

The federated model

In the federated architecture, there is no Relationship Service. Each domain service owns its own data and stores the parent reference directly (e.g., parent_product_id). Partner visibility is a column in each domain table — no separate ACL traversal needed.

Target Architecture

Minimal federated view. Domain responsibilities are grouped instead of split into many service boxes.



each domain's own store — no Relationship Service

What changes

Aspect	Before (monolith)	After (federated)
Data retrieval	Relationship Service graph traversal	Direct query per domain
ACL / partner visibility	Serial per-node ACL calls	Column filter: WHERE partner_id = ?
Latency profile	Sequential: graph ACL Elastic	Fully parallel: all domain queries run at once
Code ownership	1 function, 1 team	Each domain team owns its field resolver

Solution Options

We evaluated five approaches. **Gateway Type-Merging was evaluated and excluded** — see [Appendix: Why Type-Merging Was Excluded](#).

Option	Name	One-liner
A	Standard Federation (F4)	Each domain DGS extends ProductTechPack type. Gateway auto-routes fields.
B	Elastic Search DGS	Single DGS queries existing Elastic indices — all tech pack data is already indexed there.
C	Dedicated Orchestrator DGS	Single techpack-dgs calls domain REST endpoints in parallel.
D	Interface Technique (HasTechPackFields)	Domain DGSes implement a shared interface. TechPack DGS calls domain DGS queries via interface contract.
E	Materialized Read Model	Domain services publish events. TechPack DGS maintains a pre-computed snapshot.

Option A — Standard Federation (F4)

Concept

Each domain DGS uses extend type ProductTechPack to add the fields it owns. The gateway discovers ownership via schema introspection and fans out automatically.

OPTIONA1

Product DGS defines:

```
type ProductTechPack @key(fields: "productId partnerId") {
  productId, partnerId, workspaceContext, parentProductId
}
```

Attachment DGS:

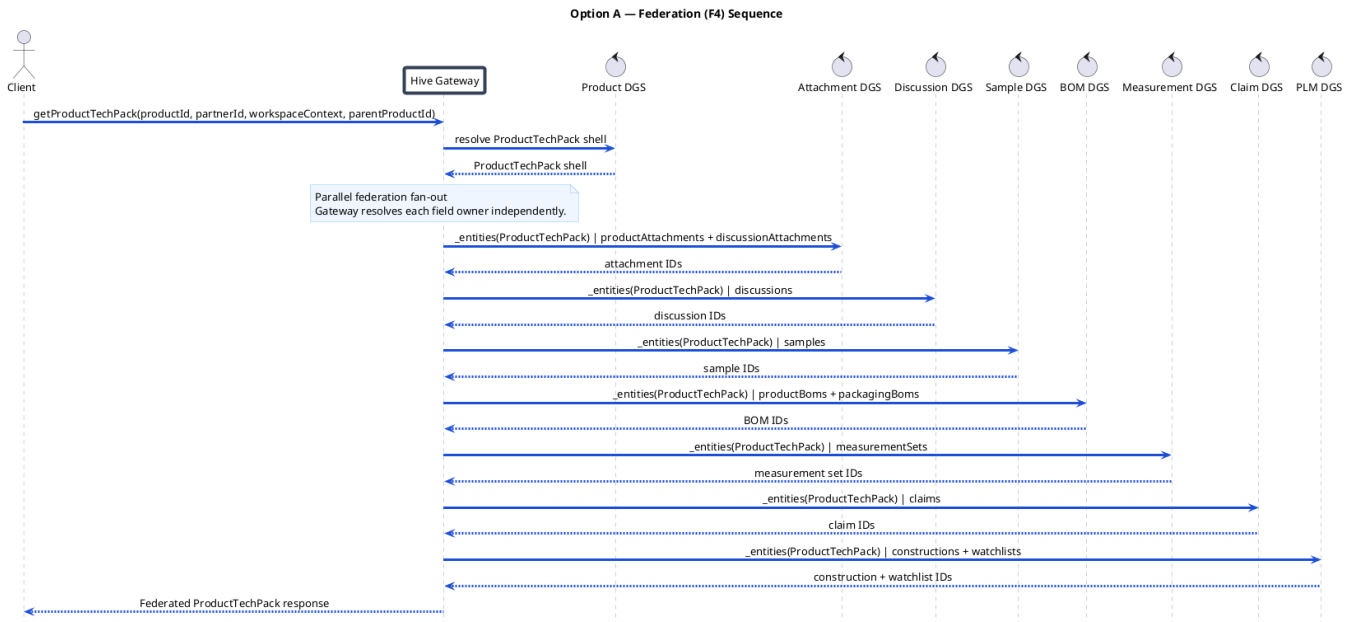
```
extend type ProductTechPack @key(fields: "productId partnerId") {
  productAttachments: [ID!]! @requires(fields: "productId partnerId parentProductId")
  discussionAttachments: [ID!]! @requires(fields: "productId partnerId parentProductId")
}
```

Sample DGS:

```
extend type ProductTechPack @key(fields: "productId partnerId") {
  samples: [ID!]! @requires(fields: "productId partnerId workspaceContext parentProductId")
}
```

... each domain DGS extends with its own field(s)

Sequence Diagram



Pseudo Code

optiona2

Product DGS – defines the aggregate shell:

```

data class ProductTechPack(
    val productId: String,
    val partnerId: String,
    val workspaceContext: String? = null,
    val parentProductId: String? = null,
)

@DgsComponent
class ProductTechPackResolver {
    @DgsQuery
    fun getProductTechPack(
        @InputArgument productId: String,
        @InputArgument partnerId: String,
        @InputArgument workspaceContext: String?,
        @InputArgument parentProductId: String?,
    ) = ProductTechPack(productId, partnerId, workspaceContext, parentProductId)

    @DgsEntityFetcher(name = "ProductTechPack")
    fun fetch(values: Map<String, Any>) = ProductTechPack(
        productId = values["productId"] as String,
        partnerId = values["partnerId"] as String,
        workspaceContext = values["workspaceContext"] as? String,
        parentProductId = values["parentProductId"] as? String,
    )
}
  
```

Sample DGS – extends with samples field:

```

@DgsComponent
class TechPackSamplesResolver(private val sampleRepo: SampleRepository) {

    @DgsEntityFetcher(name = "ProductTechPack")
    fun fetch(values: Map<String, Any>) = ProductTechPack(/* same as above */)

    @DgsData(parentType = "ProductTechPack", field = "samples")
    fun samples(dfe: DgsDataFetchingEnvironment): List<String> {
        val pack = dfe.getSource<ProductTechPack>()
    }
}
  
```

```

        val productIds = listOfNotNull(pack.productId, pack.parentProductId)
        return sampleRepo.findTechPackSamples(
            productIds = productIds,
            partnerId = pack.partnerId.toInt(),
            workspaceContext = pack.workspaceContext,
        )
    }
}

```

-- **Sample DGS query**

```

SELECT s.id FROM samples s
WHERE s.parent_product_id IN (:productIds)
  AND s.partner_id = :partnerId
  AND s.evaluation_status IN (101, 102)
  AND (
    :workspaceContext IS NULL
    OR s.workspace_context = :workspaceContext
    OR s.sample_type_code IN (200, 135)
  );

```

Attachment DGS – extends with productAttachments and discussionAttachments:

```

@dgsData(parentType = "ProductTechPack", field = "productAttachments")
fun productAttachments(dfe: DgsDataFetchingEnvironment): List<String> {
    val pack = dfe.getSource<ProductTechPack>()
    val productIds = listOfNotNull(pack.productId, pack.parentProductId)
    return attachmentRepo.findCriticalProductPacketAttachments(
        productIds = productIds,
        partnerId = pack.partnerId.toInt(),
    )
}

```

-- **productAttachments

```

SELECT a.id FROM attachments a
JOIN attachment_packet_props pp ON a.id = pp.attachment_id
WHERE a.parent_resource_id IN (:productIds)
  AND a.parent_resource_type = 'product'
  AND pp.partner_id = :partnerId
  AND pp.critical = true
  AND a.media_type != '3D'
  AND a.archived = false;

```

discussionAttachments – dual-source query:

-- Source 1: attachments on discussions/threads with product_packet_props.critical=true

```

SELECT a.id FROM attachments a
JOIN attachment_packet_props pp ON a.id = pp.attachment_id
WHERE a.parent_resource_id IN (
    SELECT d.discussion_id FROM discussions d
    WHERE d.related_resource_id IN (:productIds)
    UNION
    SELECT t.thread_id FROM discussion_threads t
    JOIN discussions d ON t.discussion_id = d.discussion_id
    WHERE d.related_resource_id IN (:productIds)
)

```

```

AND pp.partner_id = :partnerId AND pp.critical = true
AND a.media_type != '3D' AND a.archived = false

```

UNION

-- Source 2: all non-3D attachments on critical discussions/threads

```

SELECT a.id FROM attachments a
WHERE a.parent_resource_id IN (
    SELECT d.discussion_id FROM discussions d
    WHERE d.related_resource_id IN (:productIds)
    AND d.critical = true AND d.partner_id = :partnerId
    UNION
    SELECT t.thread_id FROM discussion_threads t
    JOIN discussions d ON t.discussion_id = d.discussion_id
    WHERE d.related_resource_id IN (:productIds)
    AND t.critical = true AND d.partner_id = :partnerId
)

```

```
)  
AND a.media_type != '3D' AND a.archived = false;
```

Pros & Cons

Pros	Cons
True federation — each team owns and deploys their field independently	7 DGSes must all extend the same ProductTechPack type — tight schema contract
Gateway auto-discovers routing — zero manual wiring	@requires on every field means all 4 key fields are sent to every DGS — even if unused
Parallel execution is automatic	If one DGS is slow, the entire query waits
Adding a new field = new DGS extends the type — no other changes	Debugging requires tracing across 7 services
Industry-standard Apollo/DGS federation pattern	Bulk query (getProductTechPackBulk) requires _entities batching — more complex

Option B — Elastic Search DGS

Concept

The current monolith **already queries Elastic Search** for 7 of the 10 tech pack fields (samples, criticalDiscussions, measurementSets, claims, bombs, constructions, watchlists). The data is already indexed and the query patterns are proven. Instead of distributing queries across 7 domain DGSes, a single **TechPack DGS** queries the existing Elastic indices directly.

The key insight: **we already have this data in Elastic**. Rather than waiting for every domain service to build new SQL tables and migrate data, we can stand up a DGS that uses the same Elastic queries the monolith uses today — but without the Relationship Service traversal and serial ACL overhead.

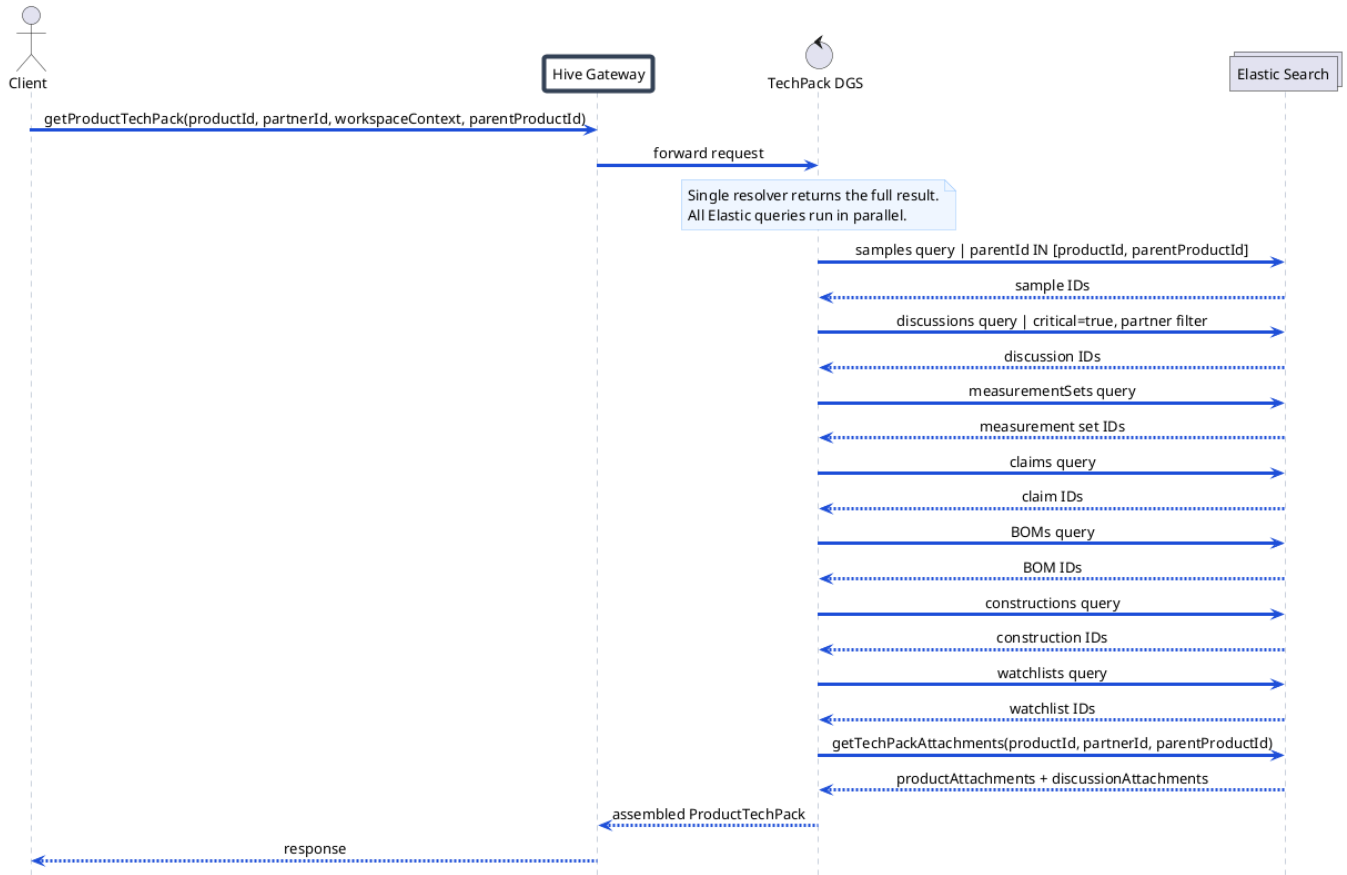
Client Gateway TechPack DGS Elastic Search (7 parallel queries)
+ Attachment Service (for product/discussion attachments)

What changes vs. the monolith

Aspect	Monolith today	Elastic DGS
Phase 1 (graph traversal + ACL)	Relationship Service serial ACL	Eliminated entirely — partner filtering via Elastic security.bps field
Phase 2 (domain queries)	7 Elastic queries in parallel	Same 7 Elastic queries — no change
Phase 3 (attachment enrichment)	getAttachmentsV3 + product_packet_props filter	Same, but can be optimized to an Elastic query
Double traversal for parentProductId	Repeat entire Phase 1	Single OR parentId:parentProductId clause in Elastic queries

Sequence Diagram

Option B - Elastic DGS Sequence



Pseudo Code

optc1

```
@DgsComponent
class TechPackElasticResolver(
    private val elasticClient: ElasticSearchClient,
    private val attachmentClient: AttachmentServiceClient,
) {
    @DgsQuery
    suspend fun getProductTechPack(
        @InputArgument productId: String,
        @InputArgument partnerId: String,
        @InputArgument workspaceContext: String?,
        @InputArgument parentProductId: String?,
    ): ProductTechPack = coroutineScope {

        .....

        .....

        ProductTechPack(
            productId          = productId,
            partnerId          = partnerId,
            workspaceContext   = workspaceContext,
            productAttachments = attachments.productAttachments,
            discussionAttachments = attachments.discussionAttachments,
            discussions        = (parentDiscussionIds + criticalDiscussionIds).distinct(),
            samples            = filteredSamples.map { it.id },
            productBoms        = boms.filter { it.type == 1 }.map { it.id },
            packagingBoms      = boms.filter { it.type == 2 }.map { it.id },
            measurementSets    = measurements.map { it.id },
            claims              = claims.map { it.id },
            constructions       = constructions.map { it.id },
            watchlists         = watchlists.map { it.id },
        )
    }

    @DgsQuery
    suspend fun getProductTechPackBulk(
        @InputArgument inputs: List<ProductTechPackInput>,
    ): List<ProductTechPack> = coroutineScope {
        inputs.map { input ->
            async {
                getProductTechPack(
                    input.productId, input.partnerId,
                    input.workspaceContext, input.parentProductId,
                )
            }
        }.awaitAll()
    }
}
```

Pros & Cons

Pros	Cons
Fastest path to production — queries already proven, data already indexed	Keeps dependency on Elastic Search cluster — not a domain-owned store
Eliminates Relationship Service + ACL traversal (the main bottleneck)	Elastic data can lag behind primary store — eventual consistency risk
Single DGS — simple deployment, single trace for debugging	TechPack DGS becomes coupled to 7 Elastic index schemas
Bulk query is straightforward — parallel coroutines	If Elastic indices change structure, TechPack DGS must update

No domain service data migration required — works with existing infrastructure	Does not advance the goal of domain-owned data stores
Partner filtering already built into Elastic security.bps field	When domain services eventually own their data, this becomes tech debt
parentProductId handling is simpler (OR clause vs. double traversal)	Attachment enrichment still needs a service call (not pure Elastic)

Option C — Dedicated Orchestrator DGS

Concept

A single techpack-dgs service owns the entire ProductTechPack type. It calls each domain service's internal REST endpoint in parallel using Kotlin coroutines. No federation, no gateway routing — the orchestrator does all aggregation.

Client Gateway techpack-dgs (parallel REST calls to domain services)

Attachment Service:

Discussion Service:

Sample Service:

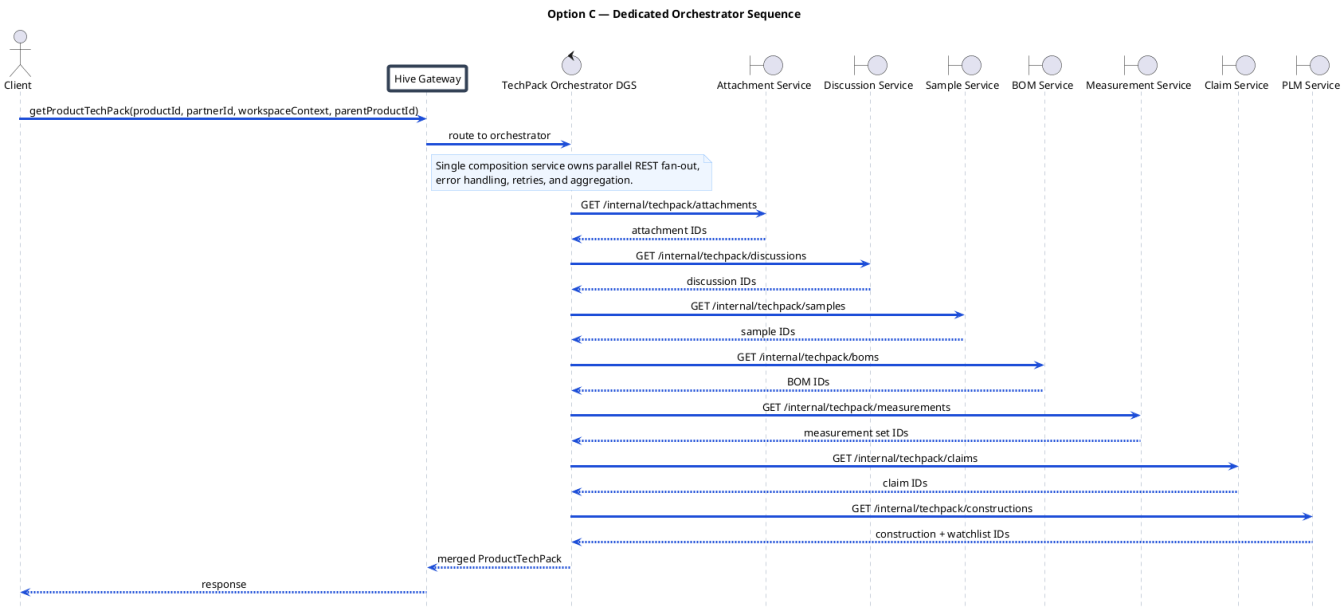
BOM Service:

Measurement Service:

Claim Service:

PLM Service:

Sequence Diagram



Pseudo Code

optc

```
@DgsComponent
class TechPackOrchestratorResolver(
    private val attachmentClient: AttachmentServiceClient,
    private val discussionClient: DiscussionServiceClient,
    private val sampleClient: SampleServiceClient,
    private val bomClient: BomServiceClient,
    private val measurementClient: MeasurementServiceClient,
    private val claimClient: ClaimServiceClient,
    private val plmClient: PlmServiceClient,
) {
    @DgsQuery
    suspend fun getProductTechPack(
        @InputArgument productId: String,
        @InputArgument partnerId: String,
        @InputArgument workspaceContext: String?,
        @InputArgument parentProductId: String?,
    ): ProductTechPack = coroutineScope {

        val attDeferred = async { attachmentClient.getTechPackAttachments(productId, partnerId,
parentProductId) }
        val discDeferred = async { discussionClient.getTechPackDiscussions(productId, partnerId) }
        val smpDeferred = async { sampleClient.getTechPackSamples(productId, partnerId, workspaceContext,
parentProductId) }
        val bomDeferred = async { bomClient.getTechPackBoms(productId, partnerId, workspaceContext) }
        val measDeferred = async { measurementClient.getTechPackMeasurements(productId, partnerId,
workspaceContext) }
        val clmDeferred = async { claimClient.getTechPackClaims(productId, partnerId, workspaceContext) }
        val plmDeferred = async { plmClient.getTechPackConstructions(productId, partnerId, workspaceContext) }

        val att = attDeferred.await()
        val disc = discDeferred.await()
        val smp = smpDeferred.await()
        val bom = bomDeferred.await()
        val meas = measDeferred.await()
        val clm = clmDeferred.await()
        val plm = plmDeferred.await()

        ProductTechPack(
            productId = productId,
            partnerId = partnerId,
            workspaceContext = workspaceContext,
            productAttachments = att.productAttachments,
            discussionAttachments = att.discussionAttachments,
            discussions = disc,
            samples = smp,
            productBoms = bom.productBoms,
            packagingBoms = bom.packagingBoms,
            measurementSets = meas,
            claims = clm,
            constructions = plm.constructions,
            watchlists = plm.watchlists,
        )
    }
}
```

Pros & Cons

Pros	Cons
Simplest to understand — one service, one resolver, explicit parallel calls	Orchestrator is tightly coupled to all 7 domain services

No federation complexity — no @requires, no _entities, no gateway config	Adding a new field = code change in orchestrator + domain service
Easy to debug — single service trace	Domains must expose a REST endpoint in addition to GraphQL
Bulk query is straightforward — loop in the orchestrator	Single point of failure — if orchestrator is down, no tech pack
Complete control over error handling, fallbacks, timeouts per call	Orchestrator team becomes bottleneck for changes

Comparison Matrix

Criteria	A: Federation (F4)	B: Elastic DGS	C: Orchestrator
Implementation effort	Medium	1 sprint	Medium
Read latency	~slowest DGS	~slowest ES query	~slowest REST call
Domain coupling	Shared schema contract	Elastic index schemas	REST code coupling
Adding new fields	Domain extends type	TechPack DGS + ES index	Orchestrator code change
Team independence	Full	Single service owns DGS	Orchestrator bottleneck
Debugging	Multi-service trace	Single service + ES	Single service trace
Consistency	Real-time	Near real-time (ES lag)	Real-time
Bulk query support	_entities batching	Parallel coroutines	Simple loop
Infra requirements	Standard DGS	Existing ES	Standard Spring Boot
Standards alignment	Apollo/DGS standard	Custom pattern	Custom pattern
Domain data migration needed	Yes – all domains	No	Yes – all domains
Relationship Service removed	Yes	Yes	Yes

Legend: = advantage = neutral = disadvantage